SANKUR KUNDU

Curriculum Vitae | ML Systems Engineer & Independent Researcher
Bhubaneswar, India (open to relocation) • +91 97761 94201 • sankur.kundu.tw@gmail.com
linkedin.com/in/sankur-kundu • github.com/Shakhtar-Sankur • shakhtar-sankur.github.io • **evidence for every claim**

7 fixes merged in 13 days at Apple, Meta, Oxford & Hugging Face • collision-risk work under review by NASA CARA

SUMMARY

ML systems engineer and independent researcher (B.Tech CSE, 2026), sole author of **four manuscripts** and co-founder of Gigzen. Seven fixes merged in 13 days into **Apple MLX**, **Meta ExecuTorch** (two), **Hugging Face OpenEnv**, **Oxford's PyZX** and **JaxMARL**. I build systems from scratch and benchmark them against the leaders: GPU kernels at **cuBLAS** parity, an LLM engine matching **vLLM** token for token, a collision-risk engine matching **NASA CARA** on all 53 of its test cases, a 494M-parameter LLM that gives **identical bits on an NVIDIA GPU and a CPU**, and a test showing the Vesuvius Challenge's official ink model finds **79% of real ink** moved into an unread Herculaneum scroll.

OPEN-SOURCE CONTRIBUTIONS: SEVEN FIXES MERGED IN 13 DAYS, 16–28 OCT 2026

APPLE MLX | Apple's ML framework for Apple silicon, **29k★** | PR #4737 **Merged 22 Oct 2026**

- Fixed a data race in the distributed ring backend, the TCP transport that links Macs for multi-machine training: its socket threads read their task queues outside the lock while other threads appended to them. A ThreadSanitizer harness found 2–4 races on every run before the fix and **0** after; MLX's ring tests pass on 8 ranks. Reviewed, approved and merged by MLX maintainer @zcbenz, who also pushed a follow-up commit (22 Oct).

META EXECUTORCH | On-device AI in Instagram & WhatsApp, **5.1k★** | #23677, #23912 **Merged 16 & 28 Oct 2026**

- The 2- and 4-bit quantized embedding kernels of on-device LLMs read memory outside the table on an invalid token id and returned it as an embedding; added the bounds check the 8-bit kernel has, with tests that fail without it (quantized suite: 82 of 82 pass). Approved by Meta maintainers Scott Roy ("LGTM! Thanks for the fix!") and Jacob Stevens; merged 16 Oct.
- **Second fix,** #23912: zero points were checked against the output type, not the scales' type, so valid fp16-scale models aborted, and the .out variants read past their scales (AddressSanitizer: heap-buffer-overflow); after Meta maintainer review, the 2- and 4-bit export kernels also report the scales' dtype. 86 of 86 kernel tests pass and the new ones fail without the fixes. Revised and rebased on review; approved and merged by Jacob Stevens, 28 Oct.

OXFORD PYZX | ZX-calculus quantum-circuit optimiser, **553★** | PR #563 **Merged 22 Oct 2026**

- Found by checking optimised circuits against the originals that three optimisers put an X rotation on the wrong qubit after rewriting two CNOTs as a SWAP, changing the unitary of **25 of 100** random Clifford circuits (0 after the fix); all 5 automated checks passed. Merged by co-creator John van de Wetering, 22 Oct: "Thanks for the fix! This was a regression from an earlier PR."

JAXMARL | NeurIPS 2024; BOLD Lab: Oxford, UCL & Imperial, **868★** | PR #206 **Merged 26 Oct 2026**

- The rollout manager behind 8 Q-learning baselines (QMIX, VDN, IQL, PQN-VDN, SHAQ, TransfQMIX) zeroed the first observation of every new episode after an auto-reset, so agents chose their first action blind and trained on it; fixed while keeping zeros for agents that die mid-episode, and two of three new tests fail without the fix. Approved and merged by lead author Alex Rutherford, 26 Oct: "Good find!!"

HUGGING FACE OPENENV | RL environments (Meta-PyTorch, NVIDIA, Microsoft), **2.8k★** | #1441, #1442 **Merged 19 Oct 2026**

- Fixed composite reward rubrics for RL training that ran pre-hooks after scoring their children (a timer read 0.01 ms for 400 ms of work) and scored an async judge as a coroutine; 9 of 13 new tests fail without the fixes. Both merged 19 Oct: "177 rubric tests pass".

In review

- **Google JAX** #41859 (**36.5k★**; Google assigned it to JAX co-author Matthew Johnson, 21 Oct): jax.grad through lax.reduce_window with a custom reduction has failed since 2021; its derivative is now a fold over the window offsets built only from transposable primitives, in the order XLA evaluates the reduction; 2,097 cases pass in 32-bit and 2,578 in 64-bit, and the 242 new ones fail on main.
- **vLLM** #60955 (**95.7k★**; labelled a bug for its NVIDIA project): FlashInfer prefill tests compared non-causal attention with a causal reference; against a float64 reference, all 32 cases now pass and all 32 fail on the bug.
- **NVIDIA Ising-Decoding** #98: the public-model tests, smoke test and docs looked only for .pt checkpoints, so anyone using the SafeTensors models NVIDIA publishes on Hugging Face failed them; they now accept the published files.
- **PyTorch** #199875, #199441, #199646: torch.compile now fills torch.empty in deterministic mode as eager does, without filling Inductor's own buffers (why an earlier fix was reverted); checked all 37 Inductor-skipped tests on CPU and two T4s, removing 13 stale skips and re-enabling 8 determinism tests, with verified causes for the 11 that stay skipped.
- **MIT Han Lab** nunchaku #957: a lone 4-bit LoRA ignored strengths below 1. **Carnegie Mellon** XGrammar #1012: the structured-output engine in vLLM and SGLang crashed when a grammar forced part of a UTF-8 character.
- **ETH Zurich** DaCe #2729: compiled numpy.rint rounded halves away from zero (2.5 → 3; NumPy gives 2); also filed a NaN bug in max/min (#2730). Maintainer: "Seems all good … once green can be merged."

Reported upstream

- **IBM Relay-BP** #47: reported that IBM's reference decoder gives different answers for the same shot depending on batch order, with a reproduction script and two candidate fixes. **Riverlane Deltakit** #411: proposed offline Relay-BP decoding for its qLDPC circuits.

HONORS & RECOGNITION

- **NASA CARA** (Conjunction Assessment Risk Analysis): my independent reproduction of CARA's Monte Carlo results on all 53 of its test cases is **under review by NASA CARA**: its Operations Team Lead referred it to CARA's analysis team, which will report its findings (20 Oct 2026; emails).
- **Entrepreneur First** (UK): its Talent Investor scouted me from public work alone (15 Oct 2026), citing my cuBLAS-parity compiler and a negative result I chose to publish, "which I rarely see" (email).

MANUSCRIPTS AND RELEASES

- S. Kundu. "Removing the Token Tax from an Existing Model: A Vocabulary Extension for 24 Indian Languages That Keeps Every Other Token." Manuscript, 2026; arXiv (cs.CL) submission in progress (paper).
- S. Kundu. "The Sampler Was Not the Policy: Finding and Removing a Silent Training–Inference Mismatch in a From-Scratch GRPO Stack." Technical report, 2026; arXiv (cs.LG) submission in progress (paper).
- S. Kundu. "Reproducing NASA CARA's Monte Carlo Probability of Collision: An Open Two-Body Implementation, Checked on All 53 of CARA's Test Conjunctions." Manuscript, 2026; arXiv (physics.space-ph) submission in progress (paper).
- S. Kundu. "Scam Defence Stops at English: Measuring the Multilingual Protection Gap and Linking Scam Campaigns Across Languages." Manuscript in preparation, 2026.
- Released lipi-Indic 96K (on Hugging Face) and samebit, both Apache-2.0.

ML SYSTEMS: TRAINING, INFERENCE AND RL

ratchet | PyTorch, C++/CUDA: reinforcement-learning post-training (GRPO) from scratch | study page

- Lifted Qwen2.5-0.5B on GSM8K from **48.1% to 52.5%** in 100 steps on 2× T4, scoring all 1,319 test problems (rollouts on relay, training on tandem).
- Cut weight sync **369×** with in-place GPU updates, and made steps **1.9×** faster with partial rollouts and one-step-ahead training.
- Found fp16 weight sync silently erasing most updates, so the sampler ran a policy the trainer never had (gap up to **5.2 nats**); mixed-precision rounding cut it to **0.04**.

relay | C++, CUDA, Swift: a disaggregated, fault-tolerant LLM inference cluster | study page

- Wrote flash-attention and flash-decoding kernels that cut 2,048-token prefill **8.7×** (3.7 s → 0.43 s) and batch-32 decode **5.4×** on a T4; output matches Hugging Face.
- Streamed the KV cache layer by layer, hiding **99–100%** of the transfer: a 46 MB hand-off went from 21.6 to **0.08 ms**.
- Kept output identical through failures: watchdogs reroute requests from killed, frozen or stalled workers (**384/384 through 60 kills**; none lost in 300 fault-injection seeds).

samebit | C, CUDA, PyTorch, MLX: LLM inference in integers, the same bits on every chip

- Ran **Qwen2.5-0.5B** (494M parameters, 24 layers, grouped-query attention) **entirely in integers** (int8 weights, int16 activations, int32 residual stream, int64 sums): WikiText-2 perplexity **23.16 vs 22.99** for float32 (**+0.75%**); 93.25% of top-1 tokens unchanged.
- Showed **bit-identity on an NVIDIA Tesla T4** (public Kaggle run): it reproduces the x86 CPU's logit digests on real text, with each dot product split over 1–64 blocks and summed by atomicAdd in a new order every run; float16 shifts logits by up to **0.25**.
- Made **six independent backends** (C99/OpenMP, NumPy, PyTorch, **Apple MLX**, **CUDA**, reference) give **one digest over all 153 intermediate tensors** on x86-64, ARM64 and the T4, checked in CI on every push.
- Proved no value overflows in any summation order (exact integer-sqrt RMSNorm; tables built with 60-digit decimals). Cost: **513–531 tokens/s** on the T4 vs 6,033–6,438 for float16 (two runs); int8 tensor-core kernels are next.

switchyard | CUDA, C++: mixture-of-experts serving with expert parallelism | study page

- Wrote routing, sorting and grouped expert GEMM/GEMV kernels on tensor cores and peer-to-peer dispatch: OLMoE-1B-7B decodes **2.2× faster than Hugging Face**, bit-identical to one GPU, with dispatch **1.25× faster than NCCL**.
- Showed on real text that dropping tokens at full experts raises perplexity from 12.9 to 20.2 even at capacity factor 2, so the kernels never drop one; found and fixed two cross-GPU stream races.

crucible | Python on Linux: sandboxed environments for RL on coding agents | study page

- Built sandboxes from namespaces, seccomp-BPF and limits that fork in **4.5 ms** (vs 138 ms to rebuild); its grader passes **0 of 9** known reward hacks (usual: 4).
- Trained with GRPO on ratchet, every reward from a sandbox: MBPP pass@1 **34.9%** → **38.6%** (3 seeds; paired test, $p = 0.013$).

tandem | C++, CUDA: distributed training without torch.distributed or NCCL | study page

- Built DDP, ZeRO-1/2/3, GPipe and 1F1B on a custom ring all-reduce that trains **bit-identically to PyTorch DDP/FSDP on NCCL**; ZeRO-3 cuts per-GPU memory 17% on an 86M-parameter GPT.
- Reached **within 2% of NCCL's** all-reduce bandwidth at 64–256 MB with lock-free rings, CUDA IPC and a fused reduce kernel.

GPU KERNELS AND COMPILERS

tessel | Rust, zero dependencies: a tile language and compiler for GPU and TPU kernels | study page

- Wrote a Triton-style compiler: typed tile IR, linear layouts, tensor-core CUDA (ldmatrix, mma.sync, swizzled shared memory, software pipelining), an auto-tuner, and Metal and TPU (Pallas) backends.
- Matched **cuBLAS** on GEMM (**1.00–1.04×**) and beat PyTorch SDPA on causal attention (**1.3–1.9×**), torch.compile on RMSNorm (1.27×) and Triton on softmax (1.07×) on a T4, checked against fp64.
- Built an LLM engine on these kernels alone (paged KV cache, continuous batching, CUDA graphs) that matches **vLLM token for token at 1.05× its throughput** on TinyLlama-1.1B.

kiln | Rust, zero dependencies: an ML compiler from ONNX to SIMD C and CUDA | study page

- Gave the lowest latency on all three benchmark models on 4 CPU cores: BERT in **117 ms** vs 149 for torch.compile, 153 for ONNX Runtime and 189 for JAX/XLA; on a T4, SmolLM2 **1.65×** faster than torch.compile.

SCIENCE: THE HERCULANEUM SCROLLS, SPACE SAFETY AND QUANTUM ERROR CORRECTION

ink-sensitivity | Python, PyTorch: calibrating ink detection for the unread Herculaneum scrolls (Vesuvius Challenge)

- Transplanted real ink from PHerc. 139, located by its published 2.4 µm ink map, into the CT volume of **PHerc. 800**, an unread scroll eligible for the challenge's prizes; the control moves bare papyrus through the same stroke shapes, so the two differ only in the ink.
- The official 9 µm model finds **79%** of the real-ink windows at a 5% false-positive rate (**AUC 0.94**; 168 ink vs 168 controls, no untouched window flagged), and transplanted ink scores like the same text left in place.

- Found that reading the volume in reversed depth order leaves the model nearly blind (**0 of 48** real-text windows), so averaging both directions dilutes it; a pure brightness profile of ink, even at 4×, is barely seen.

kessler | C++/CUDA, Python: an independent auditor of space traffic | live globe

- Matched **NASA CARA's** Monte Carlo collision probability on **all 53** of its test conjunctions (χ^2 p = 0.67), including the 28 where the standard 2D method is wrong, **15–19× faster** per core than NASA's library.
- Screens all **531 million pairs** of 32,593 tracked objects a day ahead in **28 s** on a T4, matching CelesTrak SOCRATES to 0.24 m (median); SGP4 runs 125.7M orbit states/s in fp64.
- Publishes a daily 3D globe of every active satellite's close approaches from a GitHub Actions pipeline, with a maneuver detector that uses debris as its control group.

syndrome | C++, CUDA: an open GPU decoder for IBM's quantum LDPC codes | study page

- Matched IBM's Rust Relay-BP reference **bit for bit on 256/256 shots** of the [[144,12,12]] gross code (391,320 edges), every float64 and float32 posterior included.
- Decoded a single shot **28.7× faster than IBM's decoder** (median 37.5 ms vs 1.08 s) and a batch 4.0× faster than IBM's on all CPU threads, after reproducing IBM's published error rate (41 vs 44 in 10,000).

parity | Python, Stim, PyTorch: an independent benchmark of quantum error-correction decoders

- Ran NVIDIA's two published Ising pre-decoders through NVIDIA's own pipeline (d = 5–13, 524,288 shots each), matching its logged error rates to every printed digit, against PyMatching on identical samples.
- Reproduced PyMatching 2 / Sparse Blossom (Higgott & Gidney): scaling exponent 1.38 (paper: 1.32), thresholds at 0.61–0.80% for d = 3–11, and 0 logical errors in 1M shots per basis at d = 17.

LANGUAGE AI

lipi | PyTorch: cutting the token tax on Indian languages | study page

- Measured 24 Indian languages on 15 tokenizers (Odia: 4.99× English on GPT-4o) and built lipi-Indic (on Hugging Face), which needs fewer tokens than **GPT-4o's and Gemini's in 23 of 24** (1.30× vs 1.92×, 1.58×).
- Kept English, code and other scripts on exactly their Qwen 2.5 token ids, since the 96K new merges fire only on Indian-script bytes; Santali drops from 13.7× English to 1.52×.
- Extended Qwen2.5-0.5B and continued pre-training on 2× T4: Odia takes **8.6×** fewer tokens and, in equal GPU time, is predicted **20% better**, with Odia→English chrF++ 16.0 → 24.8.

babelguard | Python: an open early-warning system for scams that cross languages (in progress) | study page

- Showed in 33,869 scam SMS across 49 languages that an English keyword filter catches **86%** of English scams but **10–36%** of Korean, Japanese, Urdu, Arabic, Thai and Hindi ones.
- Built linking by meaning that recognises a translated scam **85%** of the time (text matching: 1.5%), on a crash-safe streaming engine: **4,057 messages/s** on 4 cores, nothing lost in 50 kills.

DISTRIBUTED SYSTEMS AND VERIFICATION

quorumdb | Rust, zero dependencies: a distributed SQL database, PostgreSQL wire-compatible | study page

- Built LSM storage, self-rebalancing multi-Raft, serializable ACID transactions (Percolator) and a SQL planner that psql connects to; **1,500 tpmC** on a TPC-C-derived benchmark.
- Found **6 real bugs** with deterministic simulation (**10,000 Raft clusters, 610K crashes**, histories machine-checked) and a TLA+ model (337K states), which also catch **all 15 planted bugs**.

faultline | Rust, zero dependencies: Jepsen-style fault injection for etcd

- Checked 1.26M operations under 288 faults across 4 etcd release lines for linearizability, and traced lost updates under etcd locks to a paused leader applying a timed-out write 6 s late; fenced writes lost none.

EXPERIENCE

GIGZEN | Co-founder & Founding Engineer | Bhubaneswar, India **Apr 2026 – Present**

Early-stage startup (company name approved by India's MCA; incorporation in progress) | Sole engineer across every product

- Built **Waggle**, a delivery platform of three Android apps (riders, shops, customers) from one TypeScript/React/Capacitor codebase on Supabase PostgreSQL: **53 tables, 90 RLS policies, 154 SQL functions**.
- Moved first-accept-wins dispatch, UPI reconciliation and GST billing into the database, where no client can bypass them, and tested its access rules with **150 automated attacks**, all blocked.
- Released **Populace**, an open-source tool that fills an app with simulated users acting as real authenticated accounts through its own API (npm, **1,600+ downloads**, zero runtime dependencies, 122 self-tests); its first run found **5 defects** in 3.5 minutes after manual testing had passed, including a permission rule that silently stopped every new signup from creating a profile.
- Scaled it to **932,455 API calls** from 200 simulated drivers across 20 cities in 11 countries with no failures, and **3.17 million calls in a day** locally; pointed at Gitea's 482-operation API, it ran 1,106 calls cleanly and found 5 defects in its own adapter generator.

TECHNICAL SKILLS

GPU and compilers: CUDA (tensor cores, mma.sync, ldmatrix, graphs, IPC), NVRTC, Triton-style tile languages, Metal, Pallas, SIMD
ML systems: GRPO, DDP, ZeRO-1/2/3, FSDP, 1F1B, MoE expert parallelism, paged KV cache, int8 quantization, deterministic inference
Distributed systems: Raft, multi-Raft, MVCC, Percolator, LSM trees, deterministic simulation, TLA+, fault injection
Languages and platforms: C++20, CUDA, C, Rust, Python, Swift, TypeScript, SQL; PyTorch, JAX, MLX, PostgreSQL, Docker, Kubernetes

EDUCATION

IEM KOLKATA | B.Tech, Computer Science and Engineering (AI & ML) | Kolkata, India **Sep 2022 – Apr 2026**